```
%% exp04_all_plots_DECOMP_ALIGN_REMOVE_FULL.m
% FOM vs ROM clean vs ROM noisy vs ROM noisy-corrected
% PLUS (one realization, GAUGE-FIXED):
%   Align noisy basis to clean basis (W2-Procrustes),
%   decompose LrNa = A0 + A1 + A2 + A3,
%   test "remove-term" what-if ROMs:
%      LrNa - A1,  LrNa - A2,  LrNa - (A1+A2),  LrNa - A3
%
% Plots:
%   (1) trajectory at idx (Re) + relative errors
%   (2) projection error
%   (3) model error
%   (4) bar plot of term magnitudes: ||A1||,||A2||,||A1+A2||,||A3||
%
% Removed: eigenvector/mode-shape plots.
% Included: least stable eigenvalue info (FOM + ROM variants)

clear; close all; clc;
addpath(genpath('../1src'));

%% -------------------- SETTINGS --------------------
Re=1000; N=128; alpha=1; beta=0; meanInd=2;
dt=1e-2; K=15000;
r=25; k0=1;

sigma = 5e-8;        % std of noise in WEIGHTED space
seed  = 3001;

% which index to plot (interior index)
idx = [];            % empty -> will use mid index

%% -------------------- BUILD OS + GENERATOR --------------------
[A,M,y] = OS_Op(alpha,beta,N,Re,meanInd);
n = size(A,1);
if isempty(idx), idx = round(n/2); end

L   = -1i*(M\A);        % full generator
Phi = expm(dt*L);       % full step propagator

% Chebyshev quadrature weights (interior)
w  = clencurt_WeightsB(N);
```

```
W  = diag(sqrt(w));
W2 = diag(w);

%% -------------------- FOM TRUTH SNAPSHOTS --------------------
q0 = randn(n,1); q0 = q0/norm(q0);

Qf = zeros(n,K);
q = q0;
for k = 1:K
    q = Phi*q;
    Qf(:,k) = q;
end
t = (0:K-1)';

%% -------------------- CLEAN ROM (basis + reduced operator)↙
--------------------
rom = rom_build_clean(Qf, W, W2, L, r, alpha);
% Output struct "rom"
%   rom.U      : (n x k) POD modes (state space, unweighted)
%   rom.Ur     : (n x r) reduced basis with Ur' * W2 * Ur = I
%   rom.Lr     : (r x r) reduced generator Ur' * W2 * L * Ur
Ur0  = rom.Ur;      % clean W2-orth basis (n×r)
Uall = rom.U;       % all modes (n×Keff)

chk0 = norm(Ur0'*W2*Ur0 - eye(r),'fro');
fprintf('orth(clean)=%.2e\n', chk0);

Lr0 = Ur0' * W2 * L * Ur0;      % clean reduced generator
Pr0 = expm(dt * Lr0);           % clean reduced propagator

%% -------------------- NOISY BASIS (i.i.d. in WEIGHTED space)↙
--------------------
rng(seed);
Uw  = W * Uall;
UwN = Uw + sigma * randn(size(Uw));   % i.i.d. in weighted space
UN  = W \ UwN;

UrN = W2_orthonormalize(UN(:,1:r), W);     % W2-orthonormalize
chkN = norm(UrN'*W2*UrN - eye(r),'fro');
fprintf('orth(noisy)=%.2e\n', chkN);
```

```
LrN = UrN' * W2 * L * UrN;      % noisy reduced generator (in UrN coords)
PrN = expm(dt * LrN);           % noisy reduced propagator

%% ============================================================
%  CORRECT FIX: ALIGNMENT (W2-Procrustes) + DECOMP + REMOVALS
% ============================================================

% ---- Align UrN to Ur0 (fix gauge)
G = Ur0' * W2 * UrN;                % r×r
[U,S,V] = svd(G,'econ');
R = U*V';                           % orthogonal rotation

UrNa = UrN * R';                    % aligned noisy basis (comparable to↙
Ur0)
chkA = norm(UrNa'*W2*UrNa - eye(r),'fro');
fprintf('orth(aligned noisy)=%.2e\n', chkA);

% ---- Rotate noisy reduced generator into aligned coordinates
LrNa = R * LrN * R';                % similarity transform

% ---- Define meaningful "noise" in basis

%Neff = UrN - Ur0;
Neff = UrNa - Ur0;

% ---- Decompose LrNa = A0 + A1 + A2 + A3
A0 = Ur0'  * W2 * L * Ur0;          % clean
A1 = Neff' * W2 * L * Ur0;          % linear #1
A2 = Ur0'  * W2 * L * Neff;         % linear #2
A3 = Neff' * W2 * L * Neff;         % quadratic

Lr_sum = A0 + A1 + A2 + A3;
fprintf('\nDECOMP CHECK (aligned): ||LrNa - sum(Ak)||F = %.3e\n', norm↙
(LrNa - Lr_sum,'fro'));
fprintf('\nDECOMP CHECK (aligned): ||LrN - sum(Ak)||F = %.3e\n', norm↙
(LrN - Lr_sum,'fro'));

% ---- Magnitudes (absolute + relative)
n0  = norm(A0,'fro');
n1  = norm(A1,'fro');
n2  = norm(A2,'fro');
```

```
n12 = norm(A1+A2,'fro');
n3  = norm(A3,'fro');

fprintf('\nTERM MAGNITUDES (aligned, one realization):\n');
fprintf('||Neff||F  = %.3e\n', norm(Neff,'fro'));
fprintf('||A0||F    = %.3e\n', n0);
fprintf('||A1||F    = %.3e\n', n1);
fprintf('||A2||F    = %.3e\n', n2);
fprintf('||A1+A2||F = %.3e\n', n12);
fprintf('||A3||F    = %.3e\n', n3);

fprintf('\nRELATIVE to ||A0||F:\n');
fprintf('||A1||/||A0||    = %.3e\n', n1/max(n0,eps));
fprintf('||A2||/||A0||    = %.3e\n', n2/max(n0,eps));
fprintf('||A1+A2||/||A0|| = %.3e\n', n12/max(n0,eps));
fprintf('||A3||/||A0||    = %.3e\n', n3/max(n0,eps));

% ---- Build "remove-term" operators FROM ALIGNED NOISY OPERATOR
Lr_rmA1   = LrNa - A1;
Lr_rmA2   = LrNa - A2;
Lr_rmLIN  = LrNa - (A1 + A2);
Lr_rmQUAD = LrNa - A3;

% Corresponding propagators
PrNa      = expm(dt * LrNa);
Pr_rmA1   = expm(dt * Lr_rmA1);
Pr_rmA2   = expm(dt * Lr_rmA2);
Pr_rmLIN  = expm(dt * Lr_rmLIN);
Pr_rmQUAD = expm(dt * Lr_rmQUAD);

%% -------------------- TRACE-I CORRECTION (apply in aligned coords)↙
--------------------
trA  = trace(L);                        % A = L here
bias = (sigma^2) * trA * eye(r);
LrC  = LrNa - bias;
PrC  = expm(dt * LrC);

fprintf('\ntrace(L)=%.6g%+.6gi,  |bias|=%.3e\n', real(trA), imag(trA),↙
norm(bias,'fro'));

%% ============================================================
```

```
%  LEAST STABLE EIGENVALUE INFO (FOM + ROM variants)
% ============================================================

% ---- FOM: generalized eigenproblem A v = omega M v
omF = eig(A,M); % omega
[~,iF] = max(imag(omF));            % least stable = largest imag(omega)
om_ls = omF(iF);
fprintf('\nFOM least-stable omega: %.6g%+.6gi  (imag=%.6g)\n', real↙
(om_ls), imag(om_ls), imag(om_ls));

% ---- ROM: compare least-stable lambda (generator eigenvalues)
report_ls = @(name, Lr) report_least_stable(name, Lr, alpha);

report_ls('ROM clean   Lr0',    Lr0);
report_ls('ROM noisy   LrNa',   LrNa);
report_ls('ROM corr    LrC',    LrC);
report_ls('ROM noisy-A1',       Lr_rmA1);
report_ls('ROM noisy-A2',       Lr_rmA2);
report_ls('ROM noisy-LIN',      Lr_rmLIN);
report_ls('ROM noisy-QUAD',     Lr_rmQUAD);

%% ============================================================
%  ROLLOUTS
% ============================================================
Qr0       = roll_Pr(Pr0,      Ur0,  W2, Qf, k0);   % clean ROM (clean↙
basis)

% Use aligned basis/operator for all noisy-variants
QrN       = roll_Pr(PrNa,     UrNa, W2, Qf, k0);   % noisy (aligned↙
coords)
QrC       = roll_Pr(PrC,      UrNa, W2, Qf, k0);   % corrected (aligned↙
coords)

Qr_rmA1   = roll_Pr(Pr_rmA1,  UrNa, W2, Qf, k0);
Qr_rmA2   = roll_Pr(Pr_rmA2,  UrNa, W2, Qf, k0);
Qr_rmLIN  = roll_Pr(Pr_rmLIN, UrNa, W2, Qf, k0);
Qr_rmQUAD = roll_Pr(Pr_rmQUAD,UrNa, W2, Qf, k0);

%% ============================================================
%  ERRORS
% ============================================================
```

```
% Projection errors: depend only on basis (best fit each time)
Ep0 = proj_err(Qf, Ur0,  W2, k0);
EpN = proj_err(Qf, UrNa, W2, k0);

% Model errors: depend on Pr and basis
[Em0, rho0] = model_err(Qf, Ur0,  W2, Pr0,      k0);
[EmN, rhoN] = model_err(Qf, UrNa, W2, PrNa,     k0);
[EmC, rhoC] = model_err(Qf, UrNa, W2, PrC,      k0);

[Em_rmA1,   rho_rmA1]   = model_err(Qf, UrNa, W2, Pr_rmA1,   k0);
[Em_rmA2,   rho_rmA2]   = model_err(Qf, UrNa, W2, Pr_rmA2,   k0);
[Em_rmLIN,  rho_rmLIN]  = model_err(Qf, UrNa, W2, Pr_rmLIN,  k0);
[Em_rmQUAD, rho_rmQUAD] = model_err(Qf, UrNa, W2, Pr_rmQUAD, k0);

fprintf('\nrho(Pr): clean=%.4g noisy=%.4g corr=%.4g  rmA1=%.4g rmA2=%.4g↙
rmLIN=%.4g rmQUAD=%.4g\n', ...
    rho0, rhoN, rhoC, rho_rmA1, rho_rmA2, rho_rmLIN, rho_rmQUAD);

%% ============================================================
%  PLOT 4: TERM MAGNITUDE COMPARISON
% ============================================================
figure('Color','w');
vals   = [n1 n2 n12 n3];
labels = {'||A1||','||A2||','||A1+A2||','||A3||'};
bar(vals);
set(gca,'XTick',1:numel(vals),'XTickLabel',labels);
ylabel('Frobenius norm');
title(sprintf('One realization term magnitudes (sigma=%.0e)', sigma));
grid on;
% If wide dynamic range:
 set(gca,'YScale','log');

%% ============================================================
%  PLOT 1: TRAJECTORY at idx (Re) + relative errors
% ============================================================
qf  = Qf(idx,:).';

q0i = Qr0(idx,:).';
qNi = QrN(idx,:).';
qCi = QrC(idx,:).';
```

```
qA1i = Qr_rmA1(idx,:).';
qA2i = Qr_rmA2(idx,:).';
qLNi = Qr_rmLIN(idx,:).';
qQDi = Qr_rmQUAD(idx,:).';

eps0 = 1e-14;
den_i = max(abs(qf(1)), 1e-14);
rel = @(x) abs(x - qf) ./ den_i;

e0i  = rel(q0i);
eNi  = rel(qNi);
eCi  = rel(qCi);
eA1i = rel(qA1i);
eA2i = rel(qA2i);
eLNi = rel(qLNi);
eQDi = rel(qQDi);

figure('Color','w');
tiledlayout(3,2,'Padding','compact','TileSpacing','compact');

% nexttile; hold on; grid on;
% plot(t, real(qf),  'k-',  'LineWidth',1.8, 'DisplayName','FOM');
% plot(t, real(q0i), 'b--', 'LineWidth',1.6, 'DisplayName','ROM clean');
%  plot(t, real(qNi), 'r--', 'LineWidth',1.6, 'DisplayName',sprintf('ROM↙
noisy aligned (\\sigma=%.0e)',sigma));
% plot(t, real(qCi), 'g--', 'LineWidth',1.6, 'DisplayName','ROM noisy-↙
corr');
% plot(t, real(qA1i),'m:',  'LineWidth',1.6, 'DisplayName','ROM (noisy -↙
A1)');
% plot(t, real(qA2i),'c:',  'LineWidth',1.6, 'DisplayName','ROM (noisy -↙
A2)');
% plot(t, real(qLNi),'m--', 'LineWidth',1.6, 'DisplayName','ROM (noisy -↙
(A1+A2))');
% plot(t, real(qQDi),'c--', 'LineWidth',1.6, 'DisplayName','ROM (noisy -↙
A3)');
% ylabel(sprintf('Re(q(idx=%d))', idx));
% title('Trajectory at one location');
% legend('Location','best');

nexttile; hold on; grid on;
```

```
plot(t, max(e0i,1e-16),  'b-',  'LineWidth',1.6, 'DisplayName','clean');
plot(t, max(eNi,1e-16),  'r-',  'LineWidth',1.6, 'DisplayName','noisy↙
aligned');
 plot(t, max(eCi,1e-16),  'g--', 'LineWidth',1.6, 'DisplayName','noisy-↙
corr');
% semilogy(t, max(eA1i,1e-16), 'm:',  'LineWidth',1.6,↙
'DisplayName','noisy - A1');
% semilogy(t, max(eA2i,1e-16), 'c:',  'LineWidth',1.6,↙
'DisplayName','noisy - A2');
% semilogy(t, max(eLNi,1e-16), 'm--', 'LineWidth',1.6,↙
'DisplayName','noisy - (A1+A2)');
% semilogy(t, max(eQDi,1e-16), 'c--', 'LineWidth',1.6,↙
'DisplayName','noisy - A3');
xlabel('time'); ylabel('rel error');
title('Pointwise relative error');
legend('Location','best');
set(gca,'YScale','log');


nexttile; hold on; grid on;
plot(t, max(e0i,1e-16),  'b-',  'LineWidth',1.6, 'DisplayName','clean');
%semilogy(t, max(eNi,1e-16),  'r-',  'LineWidth',1.6,↙
'DisplayName','noisy aligned');
 %semilogy(t, max(eCi,1e-16),  'g--', 'LineWidth',1.6,↙
'DisplayName','noisy-corr');
 %semilogy(t, max(eA1i,1e-16), 'm:',  'LineWidth',1.6,↙
'DisplayName','noisy - A1');
% semilogy(t, max(eA2i,1e-16), 'c:',  'LineWidth',1.6,↙
'DisplayName','noisy - A2');
 plot(t, max(eLNi,1e-16), 'm--', 'LineWidth',1.6, 'DisplayName','noisy -↙
(A1+A2)');
 %semilogy(t, max(eQDi,1e-16), 'c--', 'LineWidth',1.6,↙
'DisplayName','noisy - A3');
xlabel('time'); ylabel('rel error');
title('Pointwise relative error');
legend('Location','best');
set(gca,'YScale','log');

nexttile; hold on; grid on;
plot(t, max(e0i,1e-16),  'b-',  'LineWidth',1.6, 'DisplayName','clean');
% semilogy(t, max(eNi,1e-16),  'r-',  'LineWidth',1.6,↙
```

```
'DisplayName','noisy aligned');
%  semilogy(t, max(eCi,1e-16),  'g--', 'LineWidth',1.6,↙
'DisplayName','noisy-corr');
 %semilogy(t, max(eA1i,1e-16), 'm:',  'LineWidth',1.6,↙
'DisplayName','noisy - A1');
plot(t, max(eA2i,1e-16), 'c--',  'LineWidth',1.6, 'DisplayName','noisy -↙
A2');
% semilogy(t, max(eLNi,1e-16), 'm--', 'LineWidth',1.6,↙
'DisplayName','noisy - (A1+A2)');
% semilogy(t, max(eQDi,1e-16), 'c--', 'LineWidth',1.6,↙
'DisplayName','noisy - A3');
xlabel('time'); ylabel('rel error');
title('Pointwise relative error');
legend('Location','best');
set(gca,'YScale','log');

nexttile; hold on; grid on;
plot(t, max(e0i,1e-16),  'b-',  'LineWidth',1.6, 'DisplayName','clean');
% semilogy(t, max(eNi,1e-16),  'r-',  'LineWidth',1.6,↙
'DisplayName','noisy aligned');
%  semilogy(t, max(eCi,1e-16),  'g--', 'LineWidth',1.6,↙
'DisplayName','noisy-corr');
 plot(t, max(eA1i,1e-16), 'm--',  'LineWidth',1.6, 'DisplayName','noisy↙
- A1');
 %semilogy(t, max(eA2i,1e-16), 'c:',  'LineWidth',1.6,↙
'DisplayName','noisy - A2');
% semilogy(t, max(eLNi,1e-16), 'm--', 'LineWidth',1.6,↙
'DisplayName','noisy - (A1+A2)');
% semilogy(t, max(eQDi,1e-16), 'c--', 'LineWidth',1.6,↙
'DisplayName','noisy - A3');
xlabel('time'); ylabel('rel error');
title('Pointwise relative error');
legend('Location','best');
set(gca,'YScale','log');

nexttile; hold on; grid on;
plot(t, max(e0i,1e-16),  'b-',  'LineWidth',1.6, 'DisplayName','clean');
% semilogy(t, max(eNi,1e-16),  'r-',  'LineWidth',1.6,↙
'DisplayName','noisy aligned');
%  semilogy(t, max(eCi,1e-16),  'g--', 'LineWidth',1.6,↙
'DisplayName','noisy-corr');
```

```
 %semilogy(t, max(eA1i,1e-16), 'm:',  'LineWidth',1.6,↙
'DisplayName','noisy - A1');
 %semilogy(t, max(eA2i,1e-16), 'c:',  'LineWidth',1.6,↙
'DisplayName','noisy - A2');
% semilogy(t, max(eLNi,1e-16), 'm--', 'LineWidth',1.6,↙
'DisplayName','noisy - (A1+A2)');
 plot(t, max(eQDi,1e-16), 'r--', 'LineWidth',1.6, 'DisplayName','noisy -↙
A3');
xlabel('time'); ylabel('rel error');
title('Pointwise relative error');
legend('Location','best');
set(gca,'YScale','log');

%% ============================================================
%  PLOT 2: PROJECTION ERROR (clean basis vs noisy aligned basis)
% ============================================================
figure('Color','w'); hold on; grid on;
plot(t, max(Ep0,1e-16), 'b-',  'LineWidth',1.8, 'DisplayName','Proj err↙
(clean basis)');
plot(t, max(EpN,1e-16), 'r--', 'LineWidth',1.8, 'DisplayName','Proj err↙
(noisy  basis)');
plot(t, max(Em0,1e-16), 'g--', 'LineWidth',1.8, 'DisplayName',sprintf↙
('Model err clean (\\rho=%.4g)',rho0));

xlabel('time'); ylabel('Error');
title('Projection error');
legend('Location','best');
set(gca,'YScale','log');

%% ============================================================
%  PLOT 3: MODEL ERROR (clean vs noisy vs corrected vs removals)
% ============================================================
figure('Color','w'); hold on; grid on;
plot(t, max(Em0,1e-16), 'b-',  'LineWidth',1.8, 'DisplayName',sprintf↙
('Model err clean '));
%plot(t, max(EmN,1e-16), 'r-',  'LineWidth',1.8, 'DisplayName',sprintf↙
('Model err noisy aligned '));
%plot(t, max(EmC,1e-16), 'g--', 'LineWidth',1.8, 'DisplayName',sprintf↙
('Model err noisy-corr '));

plot(t, max(Em_rmA1,1e-16),   'm-',  'LineWidth',1.6, 'DisplayName',↙
```

```
sprintf('Model err (noisy-A1) '));
plot(t, max(Em_rmA2,1e-16),   'g--',  'LineWidth',1.6, 'DisplayName',↙
sprintf('Model err (noisy-A2) '));
%plot(t, max(Em_rmLIN,1e-16),  'm--', 'LineWidth',1.6, 'DisplayName',↙
sprintf('Model err (noisy-(A1+A2)) );
plot(t, max(Em_rmQUAD,1e-16), 'c--', 'LineWidth',1.6, 'DisplayName',↙
sprintf('Model err (noisy-A3)'));

xlabel('time'); ylabel('model error ');
title('Model error ');
legend('Location','best');
set(gca,'YScale','log');

%%

figure('Color','w'); hold on; grid on;
plot(t, EpN, 'b-', 'LineWidth',2.0, 'DisplayName','Projection error↙
noisy basis');
plot(t, EmN, 'r-', 'LineWidth',2.0, 'DisplayName',sprintf('Model error↙
(noise in modes))'));
plot(t, EmC, 'g--', 'LineWidth',2.0, 'DisplayName',sprintf('Model error↙
(noise corrected)'));

xlabel('Time');
ylabel('Error');
title('Modes identified from noisy data');
legend('Location','best');

set(gca,'YScale','log');



%% ============================================================
%  SIMPLE TRAJECTORY: FOM vs ROM vs NOISY ROM
% ============================================================

qf  = Qf(idx,:).';
q0i = Qr0(idx,:).';       % clean ROM
qNi = QrN(idx,:).';       % noisy aligned ROM

figure('Color','w'); hold on; grid on;
```

```
plot(t, real(qf),  'k-',  'LineWidth',2.0, 'DisplayName','FOM');
plot(t, real(q0i), 'g--', 'LineWidth',1.8, 'DisplayName','ROM clean');
plot(t, real(qNi), 'r--', 'LineWidth',1.8, ...
     'DisplayName',sprintf('ROM noisy (\\sigma=%.0e)',sigma));

xlabel('time');
ylabel(sprintf('Re(q(idx=%d))', idx));
title('FOM vs Clean ROM vs Noisy ROM');
legend('Location','best');


%%
%%

%%
%% ============================================================
%  A2 CHANNEL (NO LOG, NO SCALING): A2hat = (W*Ur0)' * (W*(L*Neff))
% ============================================================

X = W * Ur0;            % n×r
Y = W * (L * Neff);     % n×r

% X =  Ur0;            % n×r
% Y = (L * Neff);     % n×r

% This IS the A2 cross-term matrix (up to the same inner product you↙
use)
A2hat = X' * Y;         % r×r

fprintf('\nA2hat diagnostics (raw):\n');
fprintf('  ||X||F      = %.6e\n', norm(X,'fro'));
fprintf('  ||Y||F      = %.6e\n', norm(Y,'fro'));
fprintf('  ||A2hat||F  = %.6e\n', norm(A2hat,'fro'));
fprintf('  ||A2hat||F/(||X||F||Y||F) = %.6e\n', norm(A2hat,'fro')/max↙
(norm(X,'fro')*norm(Y,'fro'), eps));

% ---------- Plot A2hat  ----------
figure('Color','w');
tiledlayout(1,2,'Padding','compact','TileSpacing','compact');
```

```
nexttile;
imagesc(abs(X)); axis tight; colorbar;
title('|Ur| = (|W * Ur0|)');
xlabel('mode'); ylabel('state Index');

nexttile;
imagesc(abs(Y)); axis tight; colorbar;
title('|A N| = (|W * (L * Neff)|)');
xlabel('mode'); ylabel('state Index');
%%
figure('Color','w');
tiledlayout(1,1,'Padding','compact','TileSpacing','compact');
nexttile;
imagesc(abs(A2hat)); axis tight; colorbar;
title('|A2| = |(W*Ur0)^H (W*(L*Neff))| ');
xlabel('mode'); ylabel('mode');

% also show signed/phase structure if you want (still no log):
% nexttile;
% imagesc(real(A2hat)); axis image; colorbar;
% title('real(A2hat) (raw)');
% xlabel('mode'); ylabel('mode');

% ---------- Columnwise (diagonal) dot-products and cosines ----------
r = size(X,2);
dotCol = zeros(1,r);
cosCol = zeros(1,r);
for j = 1:r
    xj = X(:,j);
    yj = Y(:,j);
    dotCol(j) = abs(xj' * yj);  % raw dot magnitude (amplitude included)
    cosCol(j) = abs(xj' * yj) / max(norm(xj)*norm(yj), eps); % 0..1↙
(direction)
end

figure('Color','w');
tiledlayout(2,1,'Padding','compact','TileSpacing','compact');

% ---------- Plot A2hat  ----------
```

```
nexttile;
bar(dotCol); grid on;
xlabel('mode'); ylabel('|x_j^H y_j|');
title(' Dot product magnitude  |(W*Ur0)_j^H (W*(L*Neff))_j|');

nexttile;
bar(cosCol); ylim([0 1]); grid on;
xlabel('mode'); ylabel('cosine');
title('Per-mode cosine (direction only)');

% % ---------- Optional: see if A2 comes from off-diagonals ----------
% % Compare diagonal energy vs full energy of A2hat
% diagEnergy = norm(diag(diag(A2hat)),'fro');
% fullEnergy = norm(A2hat,'fro');
% fprintf('  diag(A2hat) energy / full = %.6e\n', diagEnergy/max↙
(fullEnergy,eps));
%
% fprintf('||A0||F = %.6e\n', norm(A0,'fro'));
% fprintf('max |A2hat_ij| = %.6e\n', max(abs(A2hat(:))));
% fprintf('max |A0_ij| = %.6e\n', max(abs(A0(:))));
%
% X  = W * Ur0;          % W*Ur
% Y  = W * (L * Neff);   % W*(L*N)
%
% figure('Color','w');
% tiledlayout(1,2,'Padding','compact','TileSpacing','compact');
%
% nexttile;
% imagesc(abs(X));
% axis tight; colorbar;
% title('|W*Ur|');
% xlabel('mode'); ylabel('state index');
%
% nexttile;
% imagesc(abs(Y));
% axis tight; colorbar;
% title('|W*(L*N)|');
% xlabel('mode');
%
% cl = [0, max([abs(X(:)); abs(Y(:))])];
%
```

```
% figure('Color','w');
% tiledlayout(1,2,'Padding','compact','TileSpacing','compact');
%
% nexttile;
% imagesc(abs(X));
% caxis(cl);
% axis tight; colorbar;
% title('|W*Ur|');
%
% nexttile;
% imagesc(abs(Y));
% caxis(cl);
% axis tight; colorbar;
% title('|W*(L*N)|');


%%

%% ================================================================
%  A1 CHANNEL
% ================================================================

NX = W * Neff;         % n×r
AU = W * (L * Ur0);    % n×r

% This IS the A1 cross-term matrix
A1hat = NX' * AU;      % r×r (complex)

fprintf('\nA1hat diagnostics (raw):\n');
fprintf('  ||NX||F      = %.6e\n', norm(NX,'fro'));
fprintf('  ||AU||F      = %.6e\n', norm(AU,'fro'));
fprintf('  ||A1hat||F   = %.6e\n', norm(A1hat,'fro'));
fprintf('  ||A1hat||F/(||NX||F||AU||F) = %.6e\n', ...
    norm(A1hat,'fro')/max(norm(NX,'fro')*norm(AU,'fro'), eps));

% ---------- Plot A1hat ----------
figure('Color','w');
tiledlayout(1,2,'Padding','compact','TileSpacing','compact');

nexttile;
imagesc(abs(NX)); axis tight; colorbar;
```

```
title('|N| = |(W*Neff)|');
xlabel('mode'); ylabel('spatial Index');

nexttile;
imagesc(abs(AU)); axis tight; colorbar;
title('|AUr| = |(W*A Ur))| ');
xlabel('mode'); ylabel('Spatial Index');
%%
figure('Color','w');
tiledlayout(1,1,'Padding','compact','TileSpacing','compact');

nexttile;
imagesc(abs(A1hat)); axis tight; colorbar;
title('|A1|');
xlabel('mode'); ylabel('mode');
%%
% ---------- Columnwise dot-products + cosines ----------
r = size(NX,2);
dotColA1 = zeros(1,r);
cosColA1 = zeros(1,r);

for j = 1:r
    nj  = NX(:,j);
    auj = AU(:,j);

    dotColA1(j) = abs(nj' * auj);
    cosColA1(j) = abs(nj' * auj) / ...
        max(norm(nj)*norm(auj), eps);
end

figure('Color','w');
tiledlayout(2,1,'Padding','compact','TileSpacing','compact');

nexttile;
bar(dotColA1); grid on;
xlabel('mode'); ylabel('|n_j^H (LUr)_j|');
title(' dot product magnitude  |(W*N)_j^H (W*(L*Ur0))_j|');

nexttile;
bar(cosColA1); ylim([0 1]); grid on;
xlabel('mode'); ylabel('cosine');
```

```
title('Per-mode cosine (direction only)');

% ---------- Diagonal energy ratio ----------
diagEnergyA1 = norm(diag(diag(A1hat)),'fro');
fullEnergyA1 = norm(A1hat,'fro');
fprintf('  diag(A1hat) energy / full = %.6e\n', ...
    diagEnergyA1/max(fullEnergyA1,eps));

% ---------- Compare magnitude vs A0 ----------
fprintf('||A0||F = %.6e\n', norm(A0,'fro'));
fprintf('max |A1hat_ij| = %.6e\n', max(abs(A1hat(:))));
fprintf('max |A0_ij| = %.6e\n', max(abs(A0(:))));


%% ==================== LOCAL HELPERS ====================

function Qr = roll_Pr(Pr, Ur, W2, Qf, k0)
K = size(Qf,2);
a = Ur' * W2 * Qf(:,k0);
Qr = zeros(size(Qf));
for k = 1:K
    Qr(:,k) = Ur * a;
    a = Pr * a;
end
end

function Ep = proj_err(Qf, Ur, W2, k0)
K = size(Qf,2);
wn = @(v) sqrt(real(v'*(W2*v)));

den = max( wn(Qf(:,k0)), eps );   % fixed reference magnitude

Ep = zeros(K,1);
for k = 1:K
    qk = Qf(:,k);
    ak = Ur' * W2 * qk;           % best-fit coords at time k
    Ep(k) = wn(qk - Ur*ak) / den; % projection residual / initial norm
end
end

function [Em, rho] = model_err(Qf, Ur, W2, Pr, k0)
```

```
K = size(Qf,2);
wn = @(v) sqrt(real(v'*(W2*v)));
rho = max(abs(eig(Pr)));

Em = zeros(K,1);
a = Ur' * W2 * Qf(:,k0);
% FIXED denominator: norm of initial FOM state
den = max( wn(Qf(:,k0)), eps );
for k = 1:K
    qk = Qf(:,k);
    Em(k) = wn(qk - Ur*a) / den;
    a = Pr * a;
end
end

function report_least_stable(name, Lr, alpha)
lam = eig(Lr);                  % generator eigenvalues
[~,ii] = max(real(lam));         % least stable in continuous time: max↙
real part
lam_ls = lam(ii);

% Useful OS-style phase speed mapping often used in your plots:
% c = (i*lambda)/alpha  (since L = -i M^{-1}A, and omega relates via↙
i*lambda)
c_ls = (1i*lam_ls)/alpha;

fprintf('%-18s: lam=%.6g%+.6gi  (Re=%.6g)   c=%.6g%+.6gi\n', ...
    name, real(lam_ls), imag(lam_ls), real(lam_ls), real(c_ls), imag↙
(c_ls));
end

%%
```